



ENGENHARIA DE SOFTWARE I

Aula 1 — Introdução à Engenharia de Software

Conceito e importância • Engenharia de Software como disciplina de engenharia • Desafios do desenvolvimento

08 de setembro de 2026

Prof. Me. Julian Rodrigues Valério

Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas — IFPI Campus Paulistana

Objetivos da aula de hoje

- 1 **Entender o que é Engenharia de Software** e por que essa área surgiu e se tornou necessária.
- 2 **Perceber a importância da Engenharia de Software** para sistemas usados por milhares de pessoas e organizações.
- 3 **Compreender por que ela é uma disciplina de engenharia** e não apenas uma forma organizada de programar.
- 4 **Identificar os principais desafios do desenvolvimento de software** e relacioná-los a situações reais do dia a dia profissional.
- 5 **Conhecer o percurso da disciplina** para saber o que esperar nas próximas aulas do semestre.

Panorama da disciplina

Cinco unidades ao longo do semestre — duas avaliações formais (Aulas 8 e 16) e um projeto prático de acompanhamento.



Hoje estamos na Unidade I — o ponto de partida para toda a disciplina.

Pergunta inicial: software é só código?

Situação

Imagine um sistema de delivery usado por 100 restaurantes e 50 mil clientes.

Além do código-fonte, o que precisa existir para esse sistema funcionar de verdade, todos os dias, sem parar?

- Requisitos — o que o sistema deve fazer
- Arquitetura e projeto — como as partes se organizam
- Testes — como verificar se funciona
- Documentação e configuração
- Infraestrutura, segurança, dados e operação
- Equipe, planejamento e comunicação

O QUE É ENGENHARIA DE SOFTWARE?

“A aplicação de uma abordagem sistemática, disciplinada e quantificável ao desenvolvimento, operação e manutenção de software.”

Sistemática

Segue uma abordagem organizada, não a improvisação.

Disciplinada

Aplica práticas, processos e padrões consistentes.

Mensurável

Usa métricas para avaliar progresso e qualidade.

Ciclo de vida completo

Cobre construção, evolução e manutenção do produto.

Por que precisamos de Engenharia de Software?

- 1 Software é crítico.** Bancos, hospitais, transporte, indústria, governo, comércio e educação dependem dele para funcionar.
- 2 Escala cresceu muito.** Projetos envolvem milhares ou milhões de linhas de código e equipes distribuídas em diferentes cidades ou países.
- 3 Mudança é constante.** Requisitos, legislação, mercado, tecnologia e expectativas dos usuários mudam continuamente.
- 4 Erros custam caro.** Falhas podem gerar prejuízo financeiro, indisponibilidade, perda de dados e riscos de segurança.
- 5 Sem processo, problemas aparecem tarde.** E quanto mais tarde são descobertos, mais caros ficam para corrigir.

Programa, software e produto de software

Programa

- Código executável que realiza uma função específica.
- Pode ser pequeno, feito para uso individual.
- Exemplo: um script que renomeia arquivos em lote.

Produto de software

- Inclui código, dados, configuração, documentação e tudo o que é necessário para uso e manutenção.
- Atende usuários reais e objetivos de negócio.
- Exemplo: um sistema de vendas usado por toda uma empresa.

Software como produto: características importantes

- É intangível — não é fabricado como um objeto físico.
- É desenvolvido/engenheirado — não é produzido por cópia industrial, como uma linha de montagem.
- Pode ter enorme complexidade interna, mesmo quando a interface parece simples.
- É altamente mutável — novas necessidades geram novas versões o tempo todo.
- A manutenção e a evolução podem durar muitos anos, às vezes décadas.

Exemplo — um aplicativo de banco continua sendo modificado por anos após seu lançamento — novas funções, correções e adaptações a normas.

Qualidade de software

Qualidade não significa apenas “ausência de bugs”. Ela tem várias dimensões:



Funcionalidade

Faz o que deveria fazer?



Confiabilidade

Continua funcionando nas situações esperadas?



Usabilidade

É fácil e adequado para o usuário?



Desempenho

Responde dentro de limites aceitáveis?



Segurança

Protege dados e operações?



Manutenibilidade

É possível modificar com segurança?



Portabilidade

Pode ser executado em diferentes ambientes?

Exemplo em sala: escolha entre dois sistemas

Sistema A

- Muito rápido
- Interface bonita
- Perde pedidos ocasionalmente

Sistema B

- Mais lento
- Interface simples
- Registra corretamente todos os pedidos

Pergunta: qual é “melhor”?

Resposta — depende dos requisitos e do contexto. Para um sistema de pedidos, confiabilidade e integridade costumam pesar mais do que velocidade extrema — qualidade precisa ser definida e medida em relação às necessidades dos stakeholders.

Quem participa de um projeto de software?

Stakeholders: pessoas ou organizações afetadas pelo sistema ou que influenciam suas decisões.

- 1 Cliente/contratante** financia, solicita ou representa a necessidade do sistema.
- 2 Usuários** utilizam o sistema no dia a dia.
- 3 Analistas/engenheiros de requisitos** ajudam a compreender e especificar necessidades.
- 4 Desenvolvedores, testadores/QA e gestores** projetam, implementam, verificam qualidade e coordenam escopo, prazo, custo e riscos.
- 5 DevOps/infraestrutura e outros especialistas** apoiam a entrega e a operação contínua do sistema.

Exemplo: aplicativo de clínica

Problema — pacientes reclamam do tempo necessário para marcar consultas.

Stakeholders envolvidos:

- Pacientes
- Médicos
- Recepcionistas
- Direção e equipe de TI

Possíveis requisitos:

- Paciente agenda e cancela consultas
- Médico visualiza sua agenda
- Sistema impede conflito de horários
- Dados pessoais são protegidos

Pergunta em aberto — quem decide o que o sistema deve fazer? A definição precisa envolver os stakeholders e ser traduzida em requisitos claros — inclusive quando pacientes e médicos têm necessidades diferentes.

Por que desenvolver software é tão desafiador?

- Software é intangível — é mais difícil enxergar seu progresso e sua qualidade do que em uma obra física.
- A complexidade pode crescer rapidamente, mesmo quando a interface parece simples ao usuário final.
- Os requisitos mudam com frequência: mercado, legislação, tecnologia e usuários evoluem constantemente.
- Sistemas modernos operam em grande escala, com muitos usuários simultâneos e integrações externas.
- Equipes grandes e distribuídas tornam a comunicação e a coordenação mais difíceis.

Principais desafios enfrentados na prática

- 1 Requisitos incompletos, ambíguos ou que mudam sem controle.
- 2 Estimativas irreais de prazo e custo.
- 3 Comunicação deficiente entre equipe e stakeholders.
- 4 Arquitetura inadequada e testes insuficientes ou tardios.
- 5 Dependência excessiva de pessoas-chave e falta de acompanhamento do projeto.

Exemplo de desafio: “é só adicionar uma tela”

Pedido do cliente — “Precisamos apenas de uma tela para permitir o cancelamento do pedido.”

Perguntas que a Engenharia de Software levanta:

- Quais regras de negócio existem para o cancelamento?
- O que acontece com o pagamento já realizado?
- O que acontece com estoque ou agendamento vinculado?
- Quem tem permissão para cancelar?
- Existe prazo limite para cancelamento?
- O usuário recebe alguma notificação?
- Como testar todos os cenários possíveis?

Lição — uma mudança aparentemente pequena pode afetar várias partes do sistema — por isso a engenharia de software analisa impacto antes de implementar.

Miniestudo de caso: sistema de matrícula

A universidade lança um novo sistema de matrícula. Na primeira semana, milhares de alunos acessam simultaneamente. O sistema fica lento e alguns alunos recebem mensagens contraditórias.

- ?
 - ?
 - ?
 - ?
- É apenas um problema de programação?
- Quais requisitos de qualidade deveriam ter sido considerados (desempenho, escalabilidade)?
- Que testes poderiam ter antecipado o problema?
- Que atividades de Engenharia de Software poderiam reduzir esse risco?

“Construir software” não é apenas escrever código.

É compreender um problema, negociar necessidades, projetar uma solução, construir, testar, entregar, operar, medir e evoluir. A Engenharia de Software oferece princípios e práticas para fazer isso de forma mais previsível e com maior qualidade.

Exercícios — Parte A: conceitos

1. Explique, com suas próprias palavras, o que é Engenharia de Software.
2. Diferencie programa de produto de software, usando um exemplo.
3. Por que a Engenharia de Software é considerada uma disciplina de engenharia, e não apenas uma forma de programar?
4. Cite três desafios do desenvolvimento de software discutidos na aula.
5. Explique por que qualidade de software não significa apenas ausência de erros.

Exercícios — Parte B: aplicação

1. Uma loja quer um aplicativo para registrar vendas. Identifique pelo menos 5 stakeholders.
2. Para o mesmo sistema, escreva 4 requisitos funcionais e 4 requisitos de qualidade.
3. Um cliente pede: “Quero que o sistema seja fácil de usar e rápido.” Transforme essa frase em perguntas que precisam ser esclarecidas.
4. Um sistema funciona corretamente, mas uma alteração simples exige modificar dezenas de arquivos. Que atributo de qualidade está relacionado a esse problema? Explique.
5. Dê um exemplo de como uma falha de comunicação entre stakeholders e equipe pode gerar um defeito no software.

Exercício-desafio: análise de projeto

Cenário — uma empresa desenvolveu um sistema de pedidos em 3 meses. O sistema foi entregue no prazo, mas apresenta problemas.

- Clientes reclamam da dificuldade de uso.
- Pedidos às vezes desaparecem.
- Novas alterações quebram funcionalidades existentes.
- Somente um desenvolvedor sabe como determinada parte do sistema funciona.

Tarefa — identifique pelo menos 4 problemas de Engenharia de Software presentes nesse cenário e proponha uma prática para resolver cada um deles.

Atividade para casa: análise de um sistema conhecido

Escolha um sistema de software conhecido — banco, delivery, e-commerce, escola, clínica, transporte, etc. — e prepare uma breve análise:

- Qual problema o sistema resolve?
- Quem são seus principais stakeholders?
- Cite 5 funcionalidades do sistema.
- Cite 3 atributos de qualidade importantes para esse sistema.
- Que problemas poderiam ocorrer se o sistema fosse mal projetado?

Entrega — trazer a análise por escrito para discussão no início da próxima aula.